

Desarrollo de una Arquitectura Modular para Modelos Digitales Fenomenológicos en Python: Validación, Pruebas y Replicabilidad

Samuel Aguilera^a

^aCarrera de Ingeniería Industrial, Universidad Católica Boliviana “San Pablo”, Bolivia

samuel.aguilera@ucb.edu.bo

Resumen

En este artículo se presenta la arquitectura computacional de un Modelo Digital Fenomenológico desarrollado en Python, diseñado para ser modular, validable en varias capas y replicable en distintas industrias de proceso. El sistema tiene cuatro módulos principales: un núcleo de balances fenomenológicos, un motor de restricciones de capacidad, un módulo de configuración externa con validación automática y una interfaz de visualización en Streamlit. Como caso de validación, se probó la arquitectura en una planta de Proteína Aislada de Soya (ISP) de 1000kg/h. Una campaña de Monte Carlo de 100,000 corridas sobre el diseño inicial dio solo 4,70 % de éxito operativo. Después de ajustar los equipos críticos, una segunda campaña de 300,000 corridas alcanzó 99,88 % de éxito ($3,03\sigma$). Once pruebas unitarias automatizadas verifican la integridad numérica del sistema.

Palabras Clave: Modelo Digital, Código Abierto, Python, Monte Carlo, Simulación de Procesos.

Abstract

This paper presents the computational architecture of a Phenomenological Digital Model developed in Python. The design prioritizes modularity, multi-layer validation, and replicability across process industries. The system has four decoupled modules: a phenomenological mass and energy balance kernel, an equipment constraint engine, an external configuration layer with automatic validation, and a Streamlit visualization interface. The architecture is validated through an Isolated Soy Protein (ISP) plant case study at 1,000 kg/h. A Monte Carlo campaign of 100,000 runs on the initial design yielded only 4.70% success. After resizing the bottleneck equipment, a second campaign of 300,000 runs achieved 99.88% success (3.03σ). Eleven automated unit tests verify the system's numerical integrity, and the complete source code is publicly available under MIT license.

Keywords: Digital Model, Open Source, Python, Monte Carlo, Process Simulation.

1. Introducción

Kritzinger et al. [1] definen tres niveles de integración virtual: **Modelo Digital** (representación estática sin intercambio automático de datos), **Sombra Digital** (flujo de datos del proceso al modelo en una sola dirección) y **Gemelo Digital** (flujo bidireccional con control predictivo en tiempo real). La mayoría de los trabajos publicados se enfocan en los dos niveles superiores, pero usan plataformas comerciales que hacen difícil replicarlos de forma independiente [2]. Hace falta una plataforma de código abierto para el nivel base de esta jerarquía: el Modelo Digital fenomenológico.

América Latina, y en particular Bolivia, es una región importante en la producción de soya. Santa Cruz concentra más del 90 % de la producción nacional, con una industria de derivados (harina, aceite, proteína aislada) en crecimiento. La soya es el cultivo industrial más importante de Bolivia, con más de 1.2 millones de hectáreas cultivadas [19]. Sin embargo, el uso de herramientas de simulación y gemelos digitales en la industria latinoamericana todavía es limitado. Esto se debe a que no hay plataformas accesibles, en español y que se puedan replicar sin invertir en software comercial. La arquitectura que se presenta en este trabajo busca reducir ese problema.

Este trabajo presenta una arquitectura computacional de Modelo Digital Fenomenológico que es modular, replicable y de código abierto. El sistema está en el primer nivel de la clasificación de Kritzinger: no necesita flujo de datos en tiempo real ni infraestructura IoT. Su núcleo usa ecuaciones de conservación de masa y energía, no correlaciones

empíricas ni modelos de caja negra. Esto hace que el modelo sea interpretable, fácil de depurar y se pueda adaptar a otros procesos [15].

El artículo está organizado de la siguiente forma. La Sección 2 presenta los trabajos relacionados. La Sección 3 explica la metodología de desarrollo. La Sección 4 describe la arquitectura propuesta. La Sección 5 explica los requisitos de replicabilidad. La Sección 6 muestra la verificación numérica con el caso de estudio ISP. La Sección 7 presenta las conclusiones y el trabajo futuro.

2. Trabajo Relacionado y Comparativa

2.1 Frameworks de Código Abierto

OpenTwins [3] es un framework de código abierto para gemelos digitales composicionales basado en Eclipse Ditto, con integración IoT y visualización 3D. Requiere infraestructura Kubernetes y no incluye modelos fenomenológicos.

CoFMPy [4] es un framework en Python para prototipado rápido de gemelos digitales usando co-simulación FMI, permitiendo integrar FMUs con IA. Es la alternativa tecnológicamente más cercana porque está implementada en Python puro.

DT-Forge [5] propone una arquitectura de seis capas para gemelos con telemetría en tiempo real, que requiere Docker, MQTT, InfluxDB, MongoDB y Neo4j.

UniTwin [6] presenta gemelos digitales universales en contenedores Kubernetes con énfasis en reconfigurabilidad dinámica.

TumorTwin [7] es un framework Python para gemelos digitales personalizados en oncología. Al igual que este proyecto, usa Python puro y tiene un diseño modular.

Udugama et al. [8] proponen modelos digitales de varias capas para la industria alimentaria. **Gamero et al.** [9] presentan el concepto de *Lean Digital Twin* para biorefinerías. **Verboven et al.** [10] establecen las bases conceptuales para gemelos digitales en procesamiento de alimentos.

Cuadro 1: Comparativa de Frameworks para Modelos Digitales

Framework	Python puro	Valid. Estad.	Pruebas Unit.	Modelo Fenom.	Nivel Integr.	UI	Caso Real
Este proyecto	✓	✓	✓	✓	Modelo	✓	✓
OpenTwins [3]	×	×	×	×	Gemelo	✓	✓
CoFMPy [4]	✓	×	×	FMU ext.	Sombra	GUI	✓
DT-Forge [5]	✓	×	×	Opcional	Gemelo	×	×
UniTwin [6]	×	×	×	×	Gemelo	✓	×
TumorTwin [7]	✓	×	×	✓	Modelo	×	✓

Como se muestra en la Tabla 1, la mayoría de los frameworks existentes no tienen validación estadística ni pruebas unitarias automatizadas. Nuestra propuesta es el único framework clasificado como Modelo Digital que incluye las cuatro capas de validación al mismo tiempo.

3. Metodología

El desarrollo del modelo digital siguió un proceso iterativo de cuatro etapas:

1. **Formulación fenomenológica:** Para cada operación unitaria del proceso ISP se derivaron balances de materia y energía a partir de primeros principios (conservación de masa y primera ley de la termodinámica). Las eficiencias nominales se obtuvieron de la literatura especializada [11, 12].
2. **Implementación modular en Python:** Cada etapa se programó como una función independiente (`calc_stage_N`) que recibe las variables de control y los parámetros del equipo, y devuelve un diccionario con los resultados

del balance. La función `run_process_model` conecta las etapas en secuencia.

3. **Validación y verificación por capas:** Se implementaron tres niveles: (i) validación de rangos físicos en los parámetros de entrada, (ii) once pruebas unitarias que verifican la consistencia numérica, y (iii) verificación del cierre del balance de masa global (error < 0,1 %).
4. **Análisis estocástico de robustez:** Se ejecutaron campañas de Monte Carlo para evaluar el comportamiento del sistema cuando las 19 variables de control varían simultáneamente. Los cuellos de botella se identificaron mediante el análisis de frecuencias de falla.

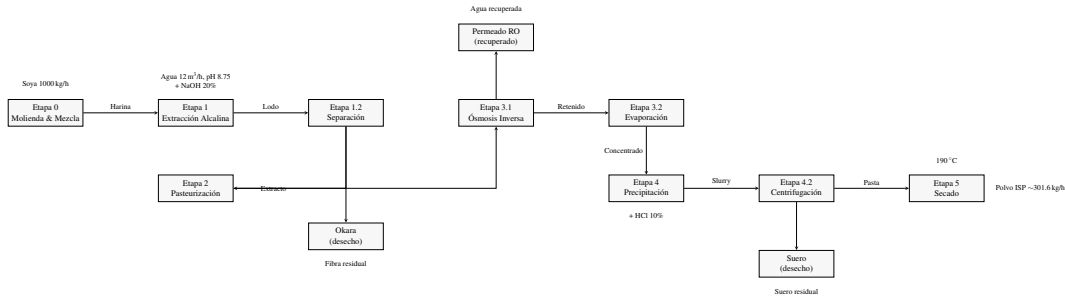


Figura 1: Diagrama de flujo del proceso ISP con las seis etapas principales.

4. Arquitectura del Modelo Digital

El sistema está compuesto por cuatro módulos computacionales en el paquete `core/` y un módulo de interfaz, todos desarrollados en Python 3.10+. Los módulos se comunican entre sí mediante diccionarios con una estructura definida.

4.1 Vista General de Módulos

- `core/equipment_specs.py` — Configuración de equipo. Contiene `EQUIPMENT_SPEC_DEFAULTS` (capacidades nominales), `EQUIPMENT_SPEC_LIMITS` (rangos físicos), `CAPACITY_LIMIT_DEFAULTS` y `CAPACITY_LIMIT_BOUNDS` (factores de seguridad). Las funciones `validate_equipment_specs` y `validate_capacity_limits` revisan los valores automáticamente.
- `core/stage_equations.py` — Núcleo fenomenológico. Las funciones `calc_stage_N` resuelven balances de materia y energía por etapa; los diccionarios `CONTROL_LIMITS` y `KPI_SPECS`; y `run_process_model` coordina la ejecución, validación y cierre del balance.
- `core/equipment_constraints.py` — Motor de restricciones. `evaluate_capacity_constraints` compara la carga con la capacidad del equipo. `enforce_capacity_constraints` lanza `EquipmentCapacityError` si hay violación.
- `core/process_model.py` — Soporte de historial. Proporciona `ControlLog` (un buffer circular de 1000 puntos) y `build_snapshot` para organizar controles, especificaciones y resultados.
- `app.py` — Interfaz en Streamlit. Incluye un bucle de simulación con paso temporal configurable, inercia de primer orden y visualización en tiempo real de KPIs y alarmas.
- `tests/` — Once pruebas unitarias distribuidas en tres archivos diferentes.

4.2 Configuración Externalizada y Validación

Todos los parámetros del proceso están definidos como constantes en los módulos, con validación automática en tres capas: valores nominales, límites físicos y límites operativos. Cada variable de control u_j tiene un rango operativo $[u_{j,\text{mín}}, u_{j,\text{máx}}]$:

$$u_j \in \mathbb{R}, \quad |u_j| < \infty, \quad u_{j,\min} \leq u_j \leq u_{j,\max} \quad \forall j \quad (1)$$

4.3 Núcleo Fenomenológico

Cada etapa modela los balances de materia y energía usando ecuaciones algebraicas. También incluye funciones de penalización cuando las condiciones se desvían de los valores nominales, un factor de pérdida operativa ($F_{\text{Loss}} = 2\%$ por etapa) y mapas de eficiencia empíricos.

4.3.1 Etapa 0: Molienda y Mezcla

$$\dot{m}_0 = \dot{m}_{\text{soya}} + \rho_w Q_w \quad (2)$$

4.3.2 Etapa 1: Lixiviación Alcalina

$$\dot{m}_{\text{prot}} = \eta_{\text{ext}} X_{\text{prot}} \dot{m}_{\text{soya}} \quad (3)$$

$$\dot{m}_{\text{okara}} = \frac{\dot{m}_{\text{soya}} (1 - \eta_{\text{ext}} X_{\text{prot}})}{1 - H_{\text{okara}}} \quad (4)$$

$$\dot{m}_{\text{ext}} = \dot{m}_0 - \dot{m}_{\text{okara}} \quad (5)$$

donde η_{ext} es modulada por funciones continuas que penalizan desviaciones de pH, temperatura, tiempo de residencia y relación sólido-líquido.

4.3.3 Etapa 2: Pasteurización HTST

$$\dot{Q}_{\text{HX}} = \dot{m}_{\text{ext}} C_{p,\text{ext}} (T_{\text{out}} - T_{\text{in}}) \quad (6)$$

$$A_{\text{HX}} = \frac{\dot{Q}_{\text{HX}}}{U \Delta T_{\text{lm}}} \quad (7)$$

4.3.4 Etapa 3: Concentración (OI + Evaporación)

La preconcentración por ósmosis inversa sigue un modelo simplificado de presión transmembrana. El evaporador al vacío remueve el agua remanente:

$$\dot{m}_{\text{perm}} = f(\Delta P, \dot{m}_{\text{ext}}) \quad (8)$$

$$\dot{Q}_{\text{ev}} = \lambda \dot{m}_{\text{vap}} = U_{\text{ev}} A_{\text{ev}} \Delta T_{\text{ev}} \quad (9)$$

4.3.5 Etapa 4: Precipitación Isoeléctrica y Centrifugación

$$\dot{m}_{\text{ppt}} = \eta_{\text{ppt}} \dot{m}_{\text{prot},4} \quad (10)$$

$$\dot{m}_{\text{ppt,húmedo}} = \frac{\dot{m}_{\text{ppt}}}{1 - H_{\text{pasta}}} \quad (11)$$

4.3.6 Etapa 5: Secado por Atomización

$$\dot{m}_{\text{powder}} = \dot{m}_{\text{ppt}} \frac{1 - X_{\text{moist,in}}}{1 - X_{\text{moist,out}}} \quad (12)$$

$$\dot{Q}_{\text{dry}} = \dot{m}_{\text{air}} C_{p,\text{air}} (T_{\text{in}} - T_{\text{out}}) + \lambda \dot{m}_{\text{evap}} \quad (13)$$

4.3.7 Inercia Dinámica

La respuesta temporal sigue una ecuación diferencial de primer orden:

$$y[k+1] = y[k] + (u[k] - y[k]) (1 - e^{-\Delta t/\tau}) \quad (14)$$

4.4 Motor de Restricciones

Las restricciones de capacidad se escriben como inecuaciones que comparan la carga solicitada con la capacidad disponible, ajustada por factores de seguridad f_s :

$$V_{TK} \leq V_{TK,max} \cdot f_{s,TK} \quad (15)$$

$$\dot{Q}_{HX} \leq A_{HX,max} \cdot U \cdot \Delta T_{lm} \cdot f_{s,HX} \quad (16)$$

$$\dot{m}_{vap} \leq \dot{m}_{vap,max} \cdot f_{s,EV} \quad (17)$$

Cuando hay una violación, el sistema genera una alarma con un código, un mensaje y una recomendación. Esto separa la detección de fallas de la lógica del proceso.

4.5 Sistema de Pruebas Unitarias

Once pruebas automatizadas verifican los siguientes aspectos: consistencia del balance de materia, rendimientos no negativos, comparación del caso base con valores de referencia ($\pm 0,5\%$), detección de bloqueos de capacidad y validación de rangos de control.

5. Requisitos de Replicabilidad

La adaptación a un nuevo proceso requiere tres pasos independientes:

1. **Mapeo de equipos:** Crear EQUIPMENT_SPEC_DEFAULTS (capacidades) y CAPACITY_LIMIT_DEFAULTS (factores de seguridad). La función `validate_equipment_specs` revisa los rangos físicos.
2. **Implementación de balances:** Escribir una función `stage_n(controls, specs)` para cada operación unitaria. Si no se necesita un modelo detallado, se pueden usar factores de rendimiento empíricos.
3. **Definición de controles:** Definir CONTROL_LIMITS con los rangos operativos. La función `validate_controls` asegura que ningún valor fuera de especificación llegue al modelo.

6. Verificación Numérica: Caso de Estudio ISP

6.1 Verificación del Balance de Materia

Con los parámetros nominales ($F_{soya} = 1000 \text{ kg/h}$, $Q_{agua} = 12 \text{ m}^3/\text{h}$, $\text{pH } 8.75$, $T_{pasteur} = 80^\circ\text{C}$, $\text{pH}_{precip} = 4,5$, $T_{secado} = 190^\circ\text{C}$), el modelo produce:

- Eficiencia de extracción: 88,0%
- Proteína precipitada: 292,3 kg/h
- Polvo proteico final: 301,6 kg/h (286,5 kg/h de proteína)
- Rendimiento global de proteína: 76,40%
- Humedad residual del polvo: 5,0%
- Cierre del balance de masa: error $< 0,1\%$

6.2 Diagnóstico Estocástico del Diseño Base

Se ejecutaron 10 lotes de Monte Carlo (100,000 corridas en total) sobre el diseño base. Solo el 4,70 % de las corridas fueron exitosas ($\sigma = \pm 0,18 \%$, IC 95 %: [4,59 %, 4,81 %]). Los cuellos de botella identificados fueron: tanque de lixiviación TK-101 (78,23 % de las fallas), tanque de agua TK-100 (69,50 %), evaporador EV-301 (57,06 %) y pasteurizador HX-201 (40,69 %).

6.3 Redimensionamiento y Validación Final

Después de ajustar los equipos críticos (Tabla 2), se ejecutó una campaña de 30 lotes (300,000 corridas) con rangos ampliados ($\pm 20 \%$ en alimentación, $\pm 25 \%$ en flujo de agua). Los resultados fueron:

- **Tasa de éxito:** 99,88 % (299,633/300,000)
- **Desviación estándar:** $\pm 0,026 \%$, IC 95 %: [99,87 %, 99,89 %]
- **Nivel Sigma:** $Z = 3,03\sigma$ (~ 1200 DPMO)
- **Fallas residuales:** Solo STAGE1_TANK_CAPACITY en 0,12 %

Cuadro 2: Límites de Capacidad (Configuración Optimizada)

Restricción	Variable	Límite Máximo
TK-100	Capacidad Geométrica	22,0 m ³
TK-101	Capacidad Geométrica	35,0 m ³
HX-201	Área de Intercambio	68,5 m ²
EV-301	Capacidad Evaporativa	16,0 m ³ /h
CF-401	Caudal Hidráulico	10,0 m ³ /h
SD-501	Capacidad Evaporativa	1000 kg/h

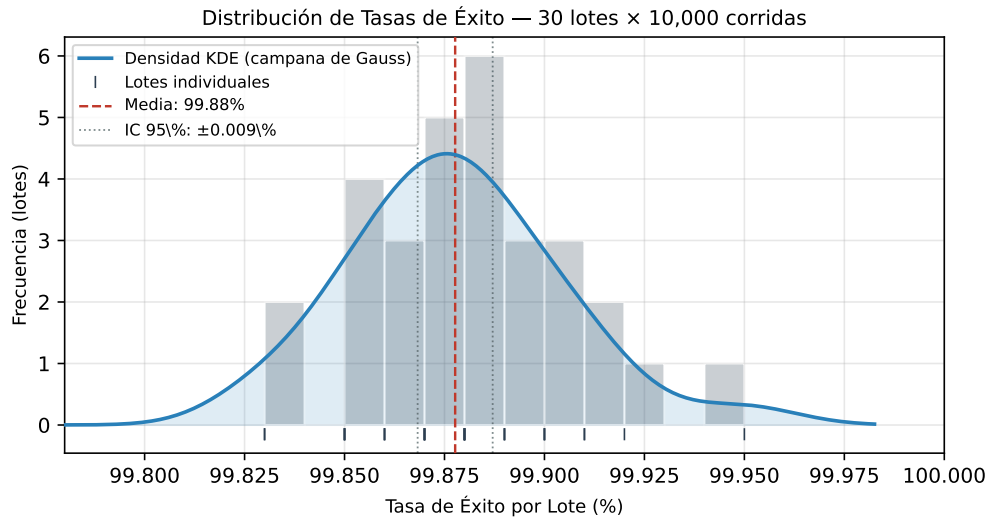


Figura 2: Distribución de tasas de éxito por lote en la campaña optimizada.

6.4 Análisis Energético

La ósmosis inversa elimina 2718,3 kg/h de agua en fase líquida usando solo 8,5 kW eléctricos. Esto reduce la carga térmica del evaporador en 836,2 kW (19,5 %). El ahorro neto de 827,7 kW permite recuperar la inversión en la membrana en menos de 1.5 años.

Cuadro 3: Comparación Energética: Sin OI vs Con OI

Variable	Sin OI	Con OI
Agua removida evap. (kg/h)	6841,0	5509,0
Agua removida OI (kg/h)	0,0	2718,3
Potencia Térmica Evap. (kW)	4294,6	3458,4
Ahorro térmico (kW)	0,0	836,2 (19,5 %)
Potencia Eléctrica OI (kW)	0,0	8,5
Ahorro Neto (kW)	0,0	827,7

7. Conclusiones y Trabajo Futuro

En este trabajo se presentó la arquitectura computacional de un Modelo Digital Fenomenológico diseñado para ser replicable en diferentes industrias de proceso. Las características principales son: (i) configuración externa con validación automática, (ii) ecuaciones independientes para cada etapa, (iii) motor de restricciones separado de la lógica del proceso, y (iv) pruebas unitarias automatizadas.

La aplicación a una planta de ISP demostró su utilidad práctica. El diagnóstico Monte Carlo mejoró el diseño de 4,70 % a 99,88 % de éxito ($3,03\sigma$), y el análisis energético mostró un ahorro del 19,5 % usando ósmosis inversa. Once pruebas unitarias garantizan la integridad numérica, y el código completo está publicado bajo licencia MIT.

El código fuente completo está disponible en un repositorio público bajo licencia MIT. Como trabajo futuro, se proyecta la transición a Sombra Digital con instrumentación IoT, la composición de múltiples plantas siguiendo el enfoque de OpenTwins y la incorporación de modelos híbridos de IA para predecir la calidad del producto.

Agradecimientos

El autor expresa su más sincero agradecimiento al Ing. Francisco Dilillo (docente de la materia de Procesos Unitarios) por su valioso apoyo y guía en la realización de este trabajo.

Referencias

- [1] W. Kritzinger et al., “Digital Twin in manufacturing: A categorical literature review and classification,” *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.
- [2] S. Gil et al., “Survey on open-source digital twin frameworks – A case study approach,” *Software Practice and Experience*, vol. 54, no. 6, pp. 929–960, 2024.
- [3] J. Robles, C. Martín, and M. Díaz, “OpenTwins: An open-source framework for the development of next-gen compositional digital twins,” *Computers in Industry*, vol. 152, p. 104007, 2023.
- [4] C. Friedrich et al., “CoFMPy: A Python Framework for Rapid Prototyping of FMI-based Digital Twins,” in *Proc. 2nd Int. Conf. Engineering Digital Twins (ICEDT)*, 2025.
- [5] DT-Forge Contributors, “DT-Forge: Domain-agnostic Python framework for digital twin architectures,” PyPI, 2025.
- [6] T. Häußermann et al., “UniTwin: Pushing Universal Digital Twins Into the Clouds Through Reconfigurable Container Environments,” *IEEE Internet Computing*, vol. 29, no. 1, pp. 8–15, 2024.
- [7] TumorTwin Contributors, “TumorTwin: A Python framework for patient-specific digital twins in oncology,” arXiv:2505.00670, 2025.
- [8] I. A. Udugama et al., “Digital twins in food processing: A conceptual approach to developing multi-layer digital models,” *Digital Chemical Engineering*, vol. 6, p. 100087, 2023.
- [9] E. Gamero et al., “Data Management in Biorefineries: Conceptual Thoughts on Lean Digital Twinning,” *Procedia CIRP*, 2024.
- [10] P. Verboven, T. Defraeye, A. K. Datta, and B. Nicolai, “Digital twins of food process operations: the next step for food process models?” *Current Opinion in Food Science*, vol. 35, pp. 79–87, 2020.
- [11] E. W. Lusas and M. N. Riaz, “Soy protein products: processing and use,” *The Journal of Nutrition*, vol. 125, no. suppl_3, pp. 573S–580S, 1995.

- [12] J. E. Kinsella, "Functional properties of soy proteins," *JAOC*S, vol. 56, pp. 242–258, 1979.
- [13] O. Kedem and A. Katchalsky, "Thermodynamic analysis of permeability," *BBA*, vol. 27, pp. 229–246, 1958.
- [14] Y. H. Roos, *Phase Transitions in Foods*, Academic Press, 1995.
- [15] R. H. Perry and D. W. Green, *Perry's Chemical Engineers' Handbook*, 9th ed., McGraw-Hill, 2019.
- [16] ISO 23247:2021, "Digital twin framework for manufacturing," ISO, 2021.
- [17] OPC Foundation, "OPC UA Companion Specification for the AAS," OPC 30270, 2022.
- [18] IEC 63278:2023, "Asset Administration Shell," IEC, 2023.
- [19] Cámara Agropecuaria del Oriente (CAO), "Informe Anual de la Soya," Santa Cruz, Bolivia, 2023.